ADR-001: LLMプロバイダー — Gemini 2.5 Flash をプライマリに採用

ステータス

承認

日付

2026-03-12

コンテキスト

シャチョツー（社長2号）は、以下のコアユースケースでLLMを大量に呼び出す設計になっている。

ユースケース	想定呼び出し頻度	トークン規模
ナレッジ抽出（テキスト → ルール/フロー/判断基準）	1社あたりオンボーディング時に数十〜数百回	入力2K〜20Kトークン
Q&Aエンジン（社長の質問に回答）	1社あたり1日10〜50回	入力1K〜5Kトークン
能動提案生成（リスクアラート・改善提案）	1社あたり1日1〜5回	入力5K〜50Kトークン
BPOプランニング（実行エージェントの計画立案）	タスクあたり1〜5回	入力1K〜10Kトークン
ゲノム合成（業界テンプレート生成・カスタマイズ）	契約時+イベント時	入力10K〜100Kトークン
推論（SaaS操作ログ → 暗黙ルール検出）	バッチ夜間	入力10K〜1Mトークン

100社規模でも月数億トークンに達する可能性がある。LLMの選択はコスト・品質・可用性の三方向でプロダクトの存続に直結する。

制約条件

1. **コスト制約**: 初期の月次ARR（売上）の10〜20%以内にAPI費用を収める必要がある。価格設定は月額5万円/社（ライト）〜30万円/社（プレミアム）を想定（ADR参照: `shachotwo/01_事業計画.md`）
2. **日本語品質**: ターゲットは日本の中小企業。業界用語（建設業・製造業・歯科等）を含む日本語テキストの読み書きが正確である必要がある
3. **長文対応**: 業務マニュアル・就業規則・過去メール等、長い文書を一括処理したいユースケースがある（コンテキストウィンドウ）
4. **スイッチング可能性**: 特定プロバイダーへの強いロックインを避けたい。LLM市場は急変動するため、将来の乗り換えコストを最小化したい
5. **1人開発チーム**: インフラ管理の負担を最小化する必要がある（オンプレ・Fine-tuningは候補外）

検討した選択肢

選択肢A: Gemini 2.5 Flash（Google AI Studio / Vertex AI）— プライマリ採用案

概要: Googleが提供するGemini 2.5 Flashモデル。速度・コスト重視のモデルで、128K（→将来1M）コンテキストウィンドウを持つ。

メリット:
- **コスト**: GPT-4oと比較して1/3〜1/5。大量呼び出しでも月次コストを許容範囲に収めやすい
- 参考: GPT-4o ≈ $5/1M入力トークン、Gemini 2.5 Flash ≈ $0.15〜$0.60/1M入力トークン（2026年3月時点）
- **コンテキストウィンドウ**: 1Mトークン（ロードマップ）。業務マニュアル全体を一度に読み込める
- **日本語品質**: GPT-4oと同等以上の日本語理解・生成品質。業界用語の認識精度も高い
- **Google Cloud統合**: Cloud Run（インフラ）・Document AI（OCR）・Whisper代替（Speech-to-Text）と同一エコシステム。IAM統合、VPC-SCによるデータ境界設定が容易
- **API安定性**: Googleの大規模インフラによる高可用性。SLA保証あり（Vertex AI利用時）
- **レイテンシ**: FlashモデルはProより応答が速く、ユーザー体験に直結するQ&Aや対話型取り込みに適している
- **マルチモーダル**: 画像・動画・音声にもネイティブ対応。将来の帳票OCR強化や動画取り込みに活用可能

デメリット:
- Googleへのベンダー依存が発生する（Google Cloudを既にインフラに採用しているため、二重依存になる）
- Claude・GPT-4oに比べて「コーディング・論理推論タスク」ではやや劣るという報告がある（ユースケースに依存）
- Vertex AI経由はGoogle Cloudプロジェクト設定が必要（初期セットアップのオーバーヘッド）

選択肢B: Claude Opus 3.5 / Sonnet 4.5（Anthropic）— フォールバック採用案

概要: Anthropicが提供するClaudeシリーズ。Sonnet系は品質・コストのバランスが良く、Opus系は最高品質。

メリット:
- **文章品質**: 日本語の自然文生成・要約・リライトはトップクラス。ナレッジ抽出や能動提案の文章クオリティが高い
- **指示遵守**: 構造化出力（JSONスキーマ準拠）の成功率が高い。Pydanticモデルへのパースエラーが少ない
- **コンテキスト理解**: 長い文脈での一貫性維持が優秀
- **Constitutional AI**: 安全性設計が組み込まれており、企業データを扱う際のリスクが低い

デメリット:
- **コスト**: Claude Sonnet ≈ $3/1M入力トークン。Gemini 2.5 Flashの5〜20倍。大量呼び出しでコスト超過リスクあり
- **API可用性**: Anthropicはレート制限が厳しい。急成長フェーズで割り当て増加の交渉コストが発生する
- **コンテキストウィンドウ**: 200K（Claude 3.5）。Gemini 1Mと比較して短い
- **Google Cloud非統合**: Vertex AIへの統合なし。Cloud RunからのAPIコール時にエグレス費用・レイテンシが増加

→ **決定**: フォールバックとして採用。品質が特に重要なユースケース（ゲノム合成・複雑な推論）ではオプションとして使用する。

選択肢C: GPT-4o / GPT-4o-mini（OpenAI）— 第3候補

概要: OpenAIが提供するGPT-4シリーズ。業界標準的な地位を持つ。

メリット:
- **エコシステム**: LangChain・LangGraph等のライブラリが最もGPT-4向けに最適化されている
- **日本語**: 十分な日本語品質
- **知名度**: 企業顧客への説明・デモが容易（「ChatGPTと同じ技術」）

デメリット:
- **コスト**: GPT-4o ≈ $5/1M入力トークン。Gemini 2.5 Flashの8〜33倍。プロダクト収益化前の段階では採算が合わない
- **コンテキストウィンドウ**: 128K。Gemini比で短い
- **OpenAI依存**: Microsoft/OpenAIへの依存。商用DPA（Data Processing Agreement）の管理コストが発生
- **API変更**: OpenAIはAPIの仕様変更・モデル廃止を頻繁に行う傾向があり、追随コストがある

→ **決定**: 第3フォールバックとして構成のみ。プライマリ・セカンダリが両方ダウンした際の緊急用。

選択肢D: ローカルLLM（Ollama + Llama 3.3 / Qwen2.5）— 却下

概要: GPUサーバーを用意し、オープンソースLLMをセルフホスト。

メリット:
- データが外部に出ない（最大のプライバシー保証）
- 変動費ゼロ（ハードウェア償却後）
- カスタムFine-tuningが可能

デメリット:
- **品質**: 日本語業界用語の理解・構造化出力の品質がGPT-4クラスより劣る。MVP品質基準を満たせないリスクが高い
- **運用コスト**: GPU（A100 8枚で100万円以上）の調達・管理・スケーリングが1人チームに不適
- **開発スピード**: インフラ構築だけで数ヶ月。MVP優先のリソース配分に反する
- **スケーリング**: 100社同時リクエストに対応するには大規模な並列インフラが必要

→ **決定**: 完全却下。将来的に特定ユースケース（ローカル推論のみ）で検討する余地はあるが、MVP範囲外。

選択肢E: Fine-tuningモデル（Custom）— 却下

概要: 自社データでFine-tuningした専用モデル。

メリット:
- 業界特化の品質向上
- レイテンシ改善の可能性

デメリット:
- **データ量**: Fine-tuningに必要な品質データが初期フェーズでは存在しない
- **コスト**: 学習コスト + 推論サーバー維持コスト
- **更新頻度**: モデル更新のたびに再Fine-tuningが必要
- **RAGで代替可能**: 業界用語・企業固有ナレッジはRAGで注入できる。Fine-tuningの優位性が薄い

→ **決定**: 完全却下。ベースモデル + RAGアーキテクチャで十分。

決定

Gemini 2.5 Flash をプライマリLLMとして採用する。

フォールバック順は以下の通り:

```
1st: Gemini 2.5 Flash       ← 全ユースケースのデフォルト
2nd: Claude Sonnet 4.5      ← Gemini障害時 / 品質最重要タスクのオプション
3rd: GPT-4o                 ← 緊急フォールバックのみ
```

モデル選択ロジック（model_tier）

`llm/client.py` に `model_tier` パラメータを設け、呼び出し元がユースケースに応じてモデルを選択できるようにする。

```
class ModelTier(str, Enum):
    FAST = "fast"         # Gemini 2.5 Flash: レイテンシ優先（Q&A, チャット）
    STANDARD = "standard" # Gemini 2.5 Pro or Claude Sonnet: バランス（抽出, 提案）
    PREMIUM = "premium"   # Claude Opus or Gemini Pro: 品質最優先（ゲノム合成, 重要判断）
```

具体的なモデルマッピングは `llm/client.py` の設定ファイルで管理し、プロバイダー乗り換え時はマッピングだけ変更すればよい設計にする。

採用理由の要約

1. **コスト**: 月100社・1000万トークン/社規模でGPT-4o比で月数百万円のコスト差が出る。プロダクトの収益性を根本的に左右する
2. **コンテキストウィンドウ**: 1Mトークンにより、大規模ドキュメント一括処理がAPIコール分割なしで可能
3. **日本語品質**: 十分な品質（GPT-4o同等以上）で業界用語対応
4. **インフラ統合**: GCP（Cloud Run）と同一エコシステム。デプロイ・認証・コスト管理が一元化できる
5. **スイッチング容易性**: `llm/client.py` 抽象化レイヤーにより、将来の乗り換えコストを最小化

影響

ポジティブな影響

- **LLMコスト削減**: GPT-4o比で70〜80%コスト削減。100社規模での月次LLMコストをARRの5〜10%以内に収められる見込み
- **長文一括処理**: ナレッジ取り込み時のチャンク分割戦略を簡略化できる
- **GCP統合**: Vertex AI経由でDocument AI・Speech-to-Text・Cloud Storageと同一認証・同一VPCで統合可能

負の影響・トレードオフ

- **Googleへの二重依存**: インフラ（Cloud Run）とLLMの両方でGoogleに依存。Googleの価格改定・規約変更の影響を直接受ける
- 緩和策: `llm/client.py` の抽象化レイヤーにより、切り替えは2時間以内で可能な設計にする
- **Gemini固有の癖**: 構造化出力（JSON mode）の挙動がOpenAI APIと異なる場合がある。`llm/client.py` で差異を吸収するアダプターが必要
- **Vertex AI設定**: Cloud RunからVertex AI APIを叩くためのIAMロール設定・サービスアカウント管理が必要

今後の制約

- 全てのLLM呼び出しは `llm/client.py` を経由すること。直接APIクライアントをインスタンス化することを禁止する
- プロンプトは `llm/prompts/` に用途別で管理し、モデル依存の記述を避ける
- モデル名のハードコードを禁止する。`ModelTier` enumを通じて間接的に指定すること
- コスト監視: Vertex AI の月次コストアラートを設定し、予算の80%到達時に通知する

マイグレーション計画

Geminiのコストが月次ARRの20%を超えた場合、または品質問題が多発した場合は、以下を評価する:
1. Gemini Pro への tier 引き上げ（コスト増・品質向上）
2. Claude Sonnet へのプライマリ切り替え（コスト増・品質向上）
3. 特定ユースケースのみローカルLLMへのオフロード

関連

- `shachotwo/02_プロダクト設計.md` — Section 3: LLM設計
- `shachotwo/01_事業計画.md` — 収支計画（LLMコスト項目）
- `llm/client.py` — 実装（LLM抽象化レイヤー）
- `llm/prompts/` — 用途別プロンプト管理
- ADR-002: Supabase採用（LLMのセマンティックキャッシュ格納先）
- ADR-004: Voyage AI採用（埋め込みモデル、LLMとは別の決定）